



ROBOT CHIRURGICAL ROSA

→ Réalisé par

RANIA MACHICHI

→ Prof Encadrant

ABDERRAHIM LAAMIMICH

PROBLÉMATIQUE

Comment le robot chirurgical ROSA utilise-t-il des systèmes d'asservissement en boucle fermée pour améliorer la précision du positionnement et la fiabilité des gestes chirurgicaux?

PLAN:

01

Introduction

Définition et classification
du robot rosa

02

Structure mécanique

Architecture et Étude du
Bras Robotisé

03

Modélisation Géométrique

Modèle direct et le modèle
inverse

04

Système d'Asservissement

Principe de la boucle
fermée et le rôle des
composants.

05

Protocole Expérimentale

Validation et Analyse
Performance



06

Analyse Du cycle De fonctionnement

Validation Expérimentale

07

CONCLUSION

Définition & Classification

ROSA ONE R Brain

Le système ROSA est un assistant robotique stéréotaxique conçu pour stabiliser et positionner des instruments lors d'interventions crâniennes. Type de robot : Cobot (Robot collaboratif) à 6 degrés de liberté, agissant comme une infrastructure guidée pilotée par logiciel.

- *Précision sub-millimétrique*
- *Navigation sans cadre (frameless stereotaxy) basée sur l'imagerie médicale (IRM/CT)*
- *Multi-trajectoires rapides et flexible*

Figure 1



PRÉSENTATION DU ROBOT ET DE SES ÉLÈMENTS

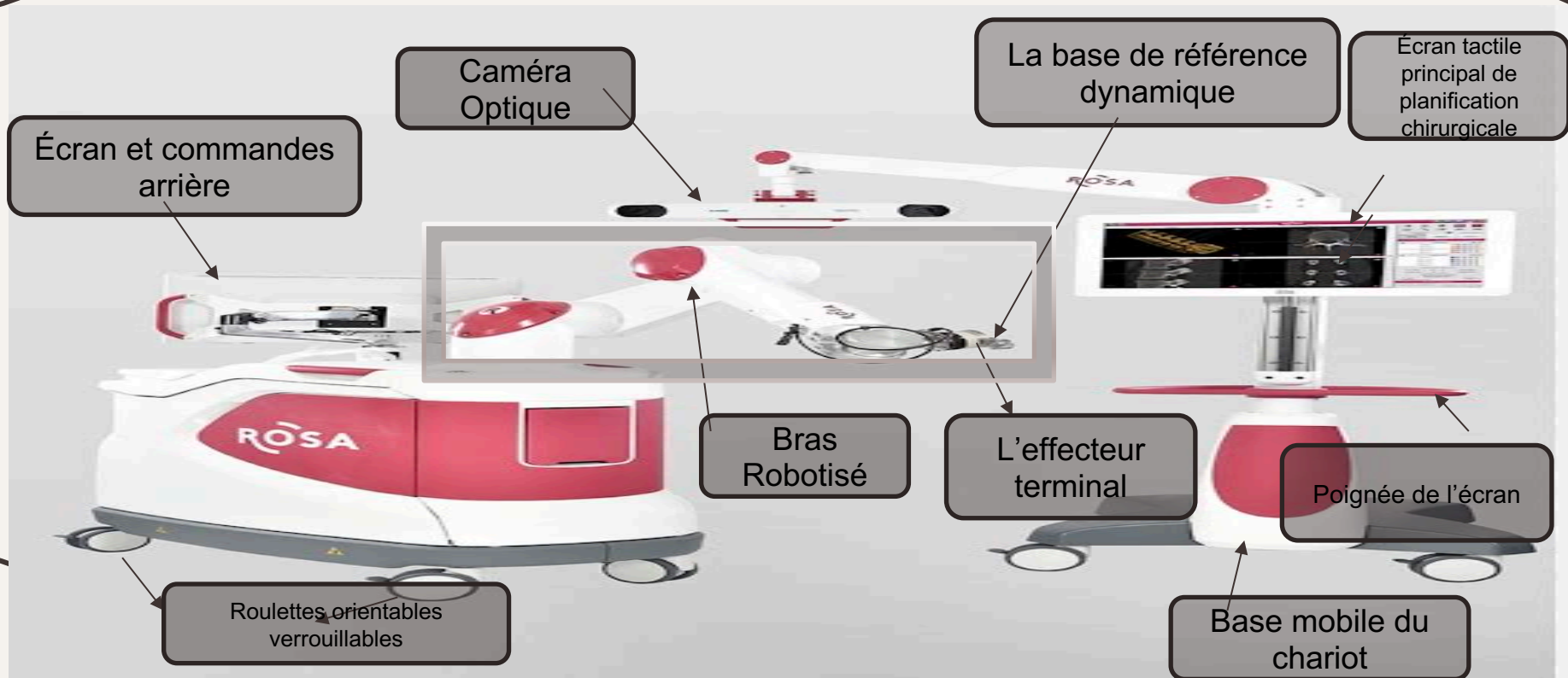


Figure 2

ETUDE DU BRAS ROBOTISÉ

MODELISATION CINÉMATIQUE

Le bras robotisé est modélisé par une chaîne cinématique reliant les mouvements articulaires à la position de l'effecteur (outil chirurgical)

Dextérité : Ses 6 articulations permettent d'atteindre une cible dans l'espace avec une orientation précise, adaptée aux contraintes stéréotaxiques en neurochirurgie.

Rigidité: Une structure rigide est essentielle pour maintenir la trajectoire malgré les forces de résistance lors du forage ou de l'insertion d'électrodes.

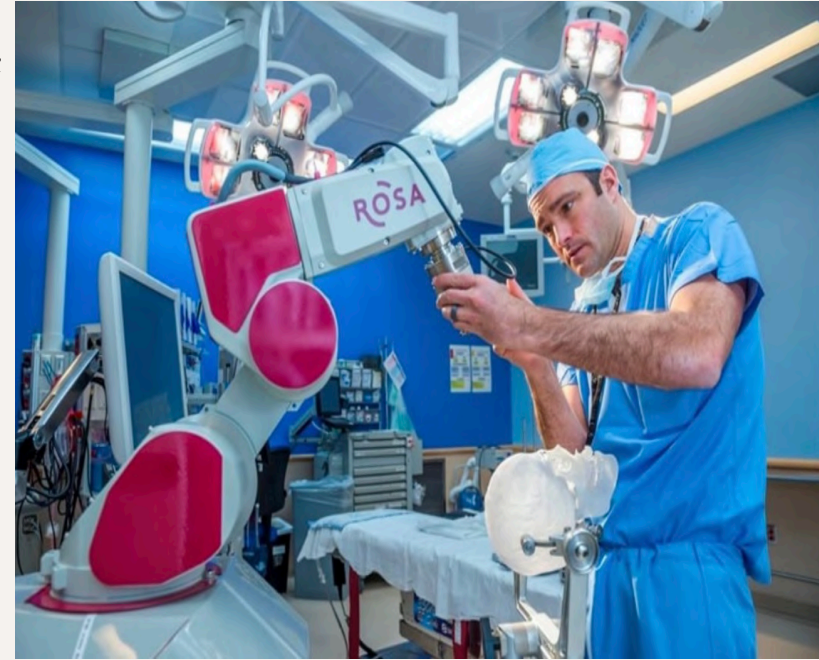


Figure 03

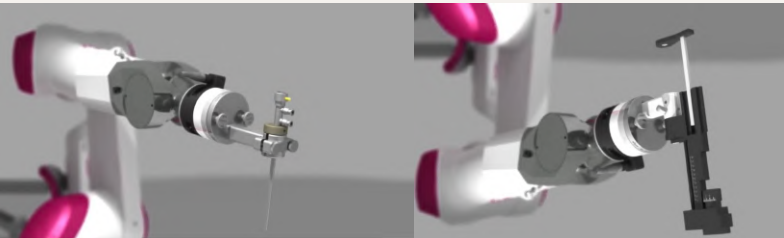


Figure 04

Applications en Neurochirurgie

Procédure Mini-invasives

°**SEEG (Épilepsie)**: Placement rapide de multiples électrodes en détection.

°**DBS(Parkinson)**: Stimulation cérébrale profonde avec guidage haute fidélité.

°**Biopsies stéréotaxiques** : Prélèvement de tissus en zones critiques.

ROSA réduit la durée opératoire de plusieurs heures par rapport aux méthodes avec cadre traditionnel.



Figure 05

BRAS ROBOTISÉ

L'objectif de ce travail est de concentrer l'analyse sur l'étude cinématique et l'asservissement du bras du robot ROSA



Figure 06

PARTIE 2

- L'ASSERVISSEMENT EN BOUCLE FERMÉE

PRINCIPE DE RÉTROACTION

ROSA fonctionne comme un système asservi : les capteurs mesurent en permanence la position réelle du bras , comparée a la consigne

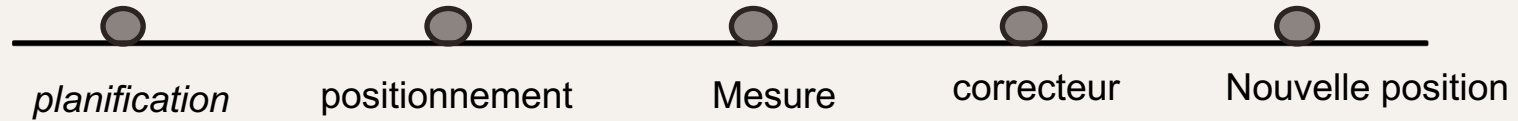
L'ÉCART (Erreur)

Si un écart est détecté entre la trajectoire planifiée et la position réelle , l'asservissement le corrige automatiquement .



Figure 07

Cycle de Fonctionnement



Ce cycle continue garantit une stabilité parfaite meme en
cas de micro - mouvements

Diagramme de Séquence : Asservissement en Boucle Fermée (Neurochirurgie)

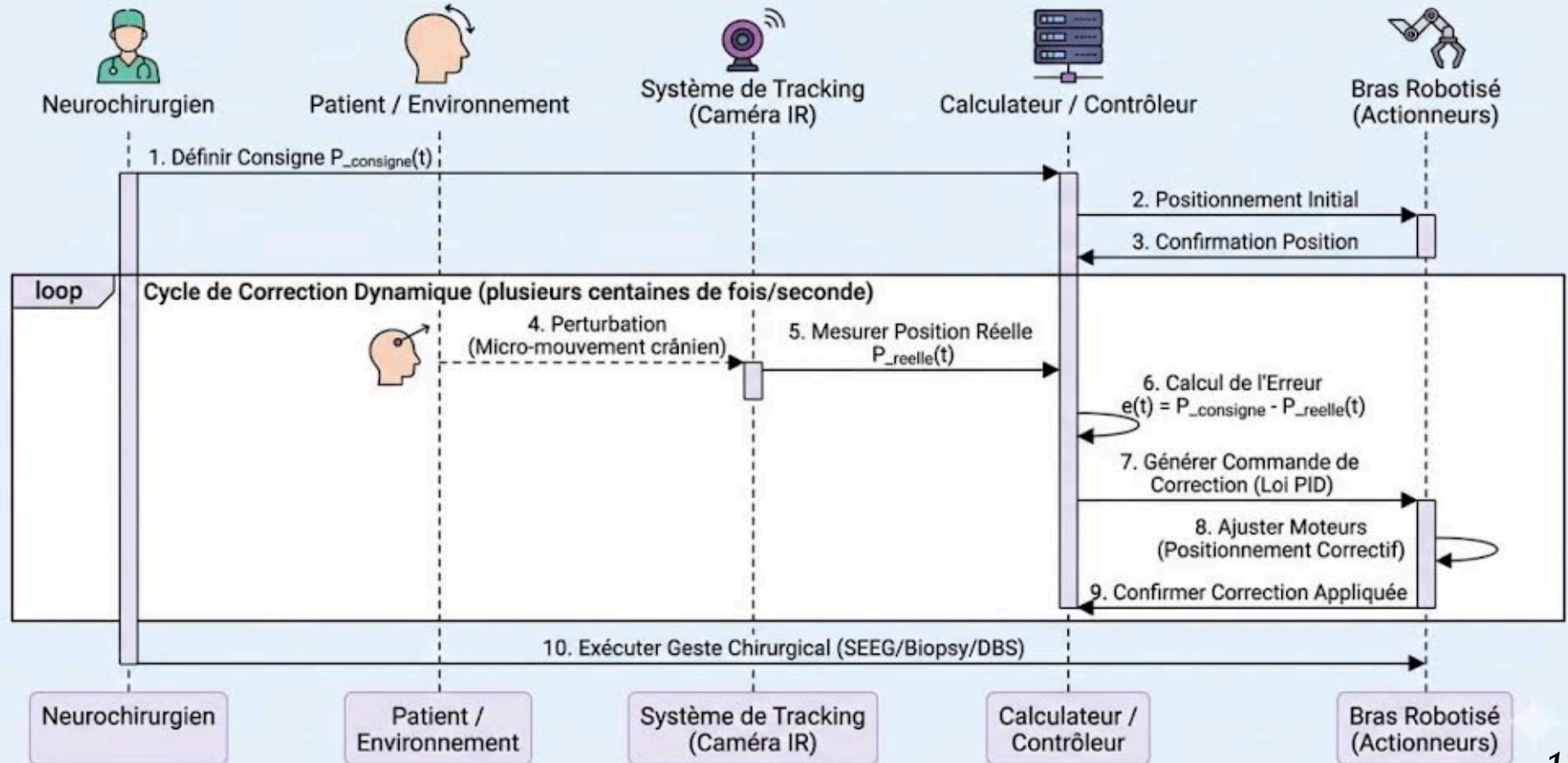


Figure 08

Diagramme de cas d'utilisation – Robot chirurgical ROSA

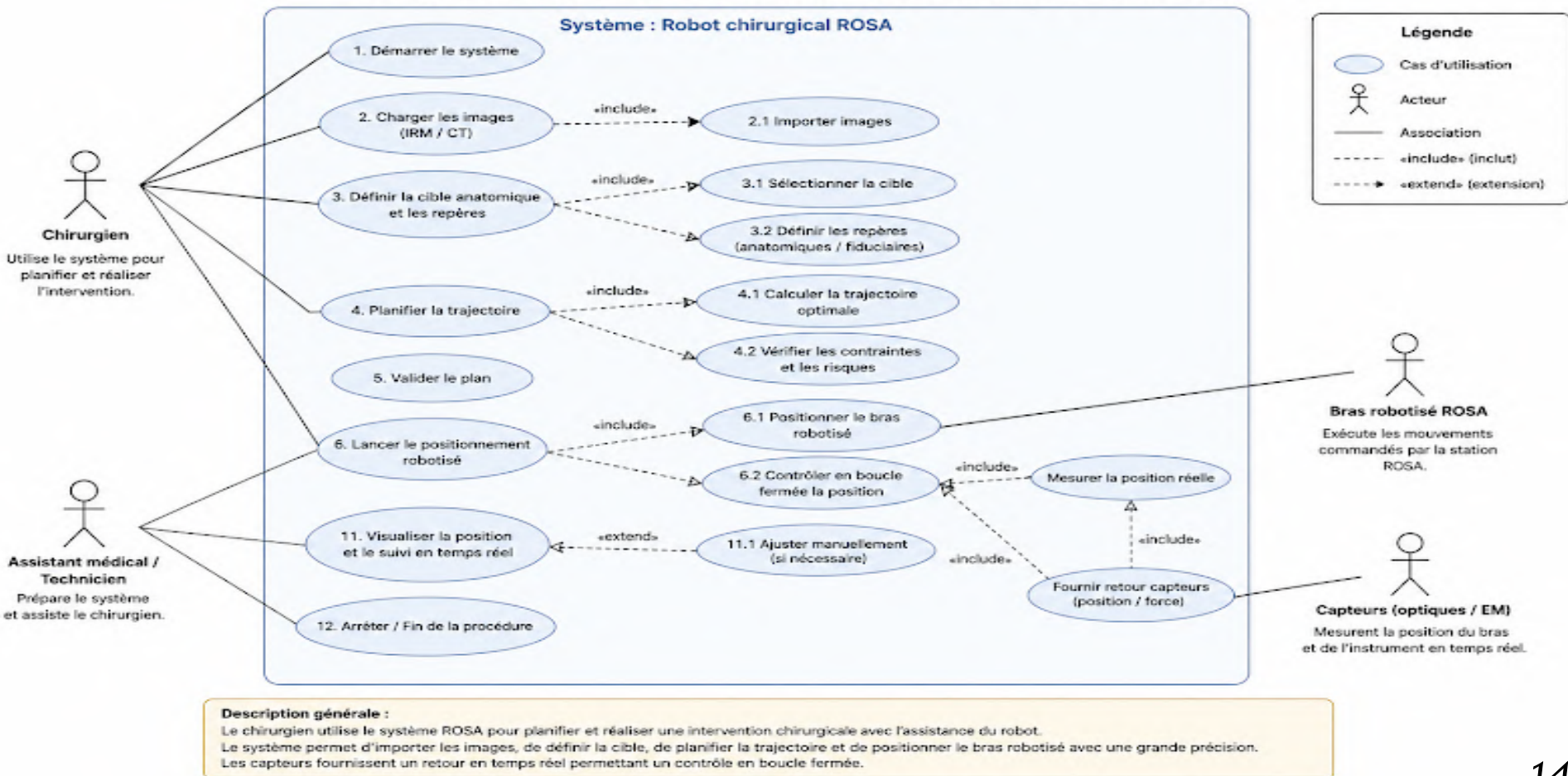


Figure 09

Modélisation simplifiée

Le système d'asservissement peut être représenté par l'équation d'erreur de position
 $e(t)$:

$$E(t) = P_{\text{consigne}}(t) - P_{\text{réelle}}(t)$$

L'asservissement minimise cet écart via une boucle de rétroaction fermée, assurant la stabilité et la fiabilité du positionnement de l'instrument chirurgical.

Mécanismes de correction:

Capteurs et rétroaction

Des capteurs de force et de couple permettent au bras de « ressentir » la résistance rencontrée.

La caméra de navigation suit en continu les repères (trackers) fixés sur le crane du patient ,permettant la correspondance entre la planification et la position réelle.

Ces informations alimentent une boucle de rétroaction (feedback) qui permet de corriger en continu la position du bras robotisé en cas de micro-mouvements du patient ou d'écarts de trajectoire.

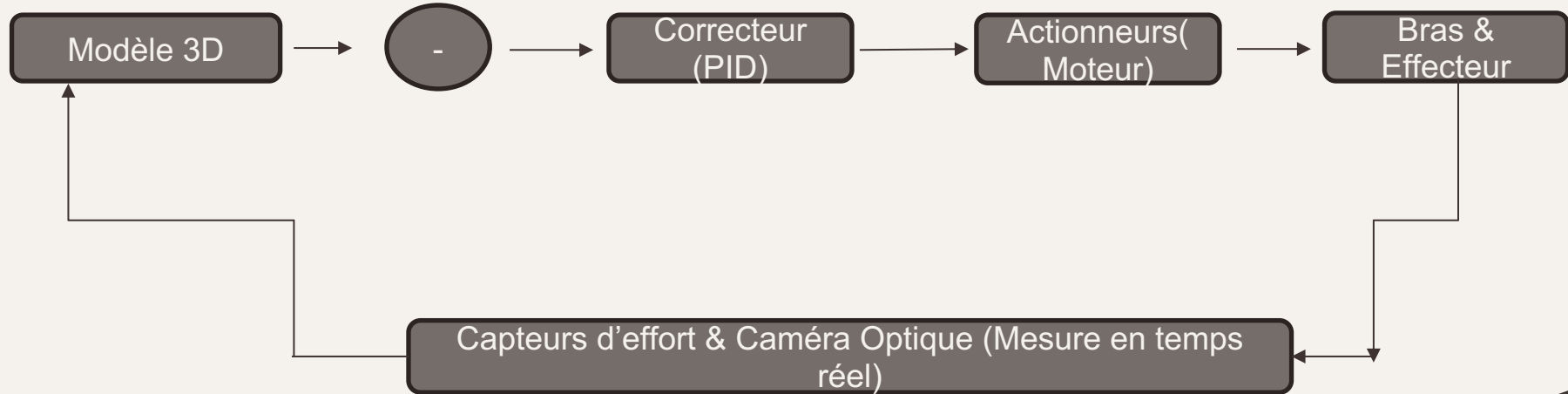
Ce système améliore la précision du geste chirurgical et réduit significativement le risque d'erreur lors des interventions intracrâniennes.



Figure 10

Schéma Bloc Boucle Fermée

Représentation de la boucle de rétroaction dynamique contrôlant le bras du robot ROSA



Le comparateur calcule en continu la différence entre l'alignement planifié et la position réelle de l'articulation du patient

CORRECTEUR PID (SIMPLIFIÉ)

Algorithme de Controle

La commande de vitesse ou d'effort appliquée aux axes du robot est régulée par une fonction de transfert PID:

$$u(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d de(t)/dt$$

K_p (proportionnel): corrige rapidement l'erreur instantanée

K_i (intégral): élimine l'erreur statique résiduelle

K_d (dérive): anticipe les variations et améliore la stabilité en réduisant les oscillations

Modèle de l'Erreur

Pour un système de positionnement à un degré de liberté (orientation de l'axe de coupe), l'erreur dynamique $e(t)$ peut être modélisée par l'équation différentielle de second ordre :

$$J \frac{d^2 e(t)}{dt^2} + B \frac{de(t)}{dt} + K e(t) = F_{\text{pert}}(t)$$

- **J (Inertie du bras)** : Résistance du robot à l'accélération angulaire.
- **B (Amortissement visqueux)** : Résistance limitant les oscillations lors du guidage.
- **K (Raideur équivalente)** : Modélise la rigidité du bras mécanique face aux pressions exercées par la scie oscillante.
- **F_pert (Force de perturbation)** : Forces résistives ou chocs générés lors de la coupe de l'os compact (cortical).

Fiabilité & Répétabilité

01

Répétabilité

Capacité à revenir exactement au même point cible pour plusieurs électrodes SEEG sans dérivation

02

Sécurité

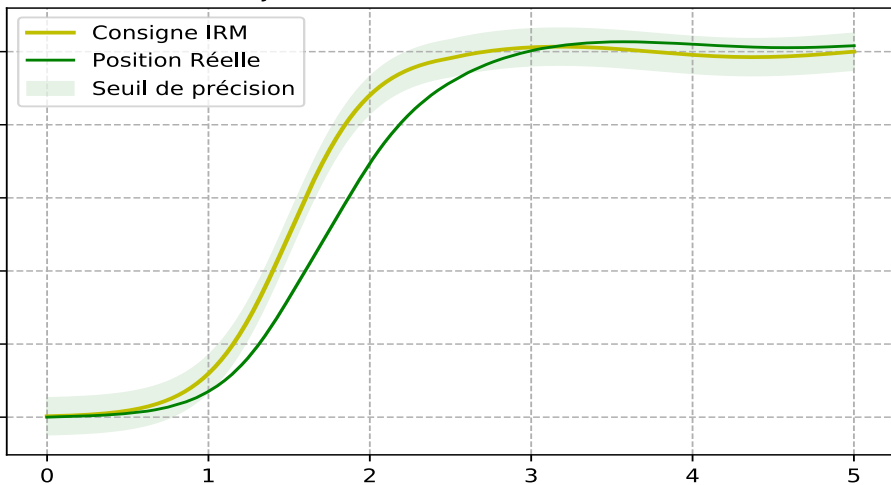
Le « cône de sécurité » logiciel bloque le robot si le chirurgien sort de la zone planifiée

03

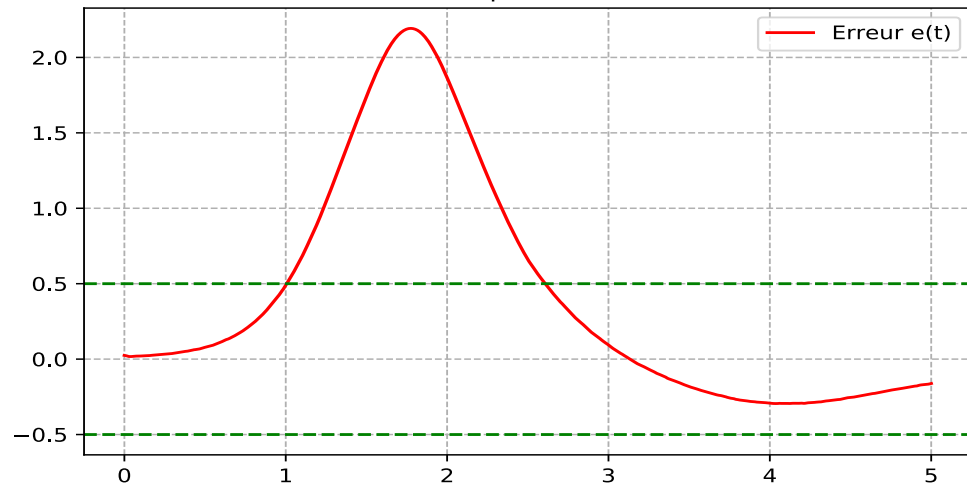
Précision

Maintien d'une marge d'erreur inférieure à 0,6mm, cruciale pour les biopsies cérébrales

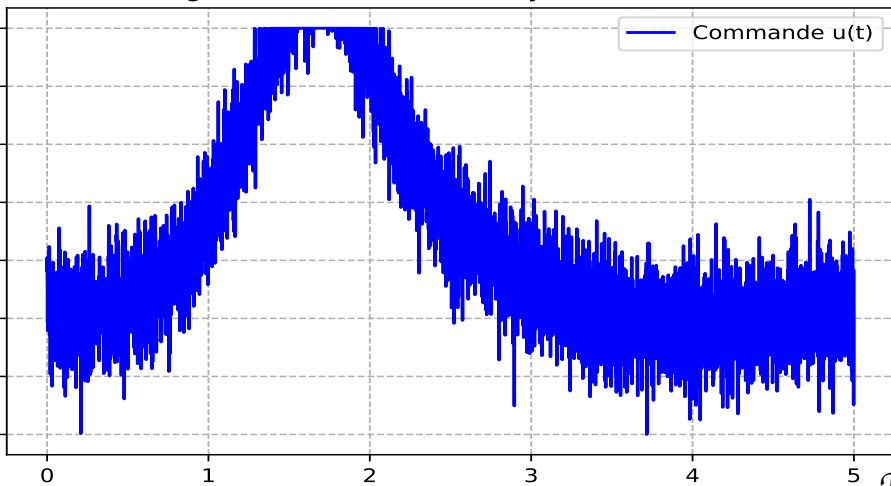
Trajectoire (Boucle fermée active)



Erreur de positionnement



Signal de commande envoyé (Actionneur)



Perturbations externes

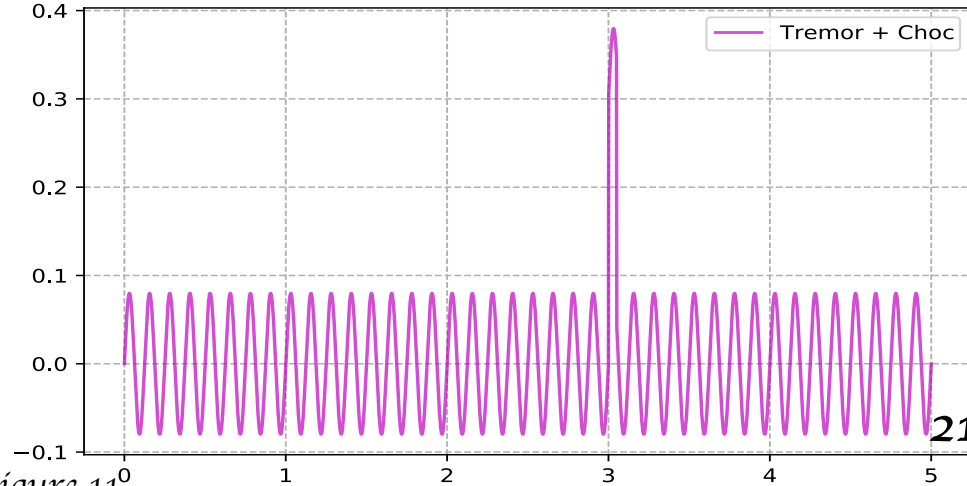


Figure 11⁰

APPROCHE EXPÉRIMENTALE

Afin de confronter ce modèle théorique à la réalité, j'ai mis en place une approche expérimentale en développant un prototype de bras robotique piloté par Arduino.



Figure 12

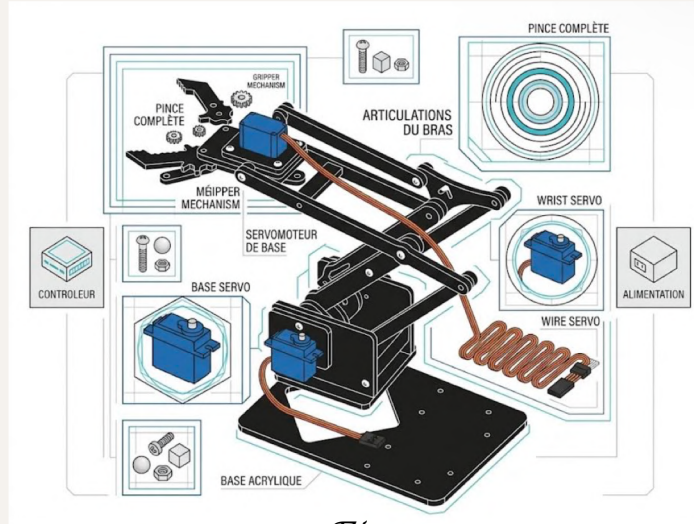


Figure 13

PRÉSENTATION DU ROBOT

Composants du robot

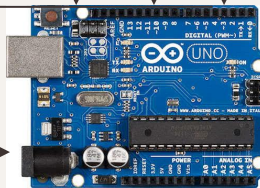


BRAS ROBOTISÉ

Shield à deux joystick pour controler le bras



Une carte arduino uno R3

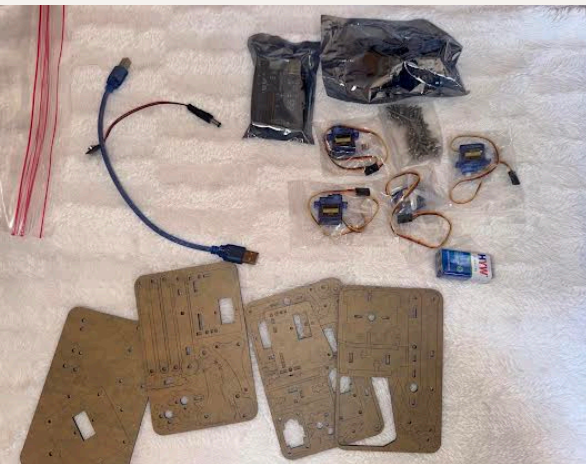


Pile 9V



MOUSSA SOFT

ÉTAPE DE L'EXPERIENCE



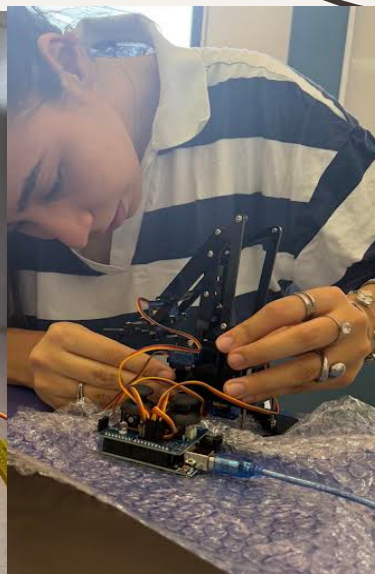
Etape 1: Rassemblement des éléments de l'expérience



Etape 2 :Construction de la structure du bras

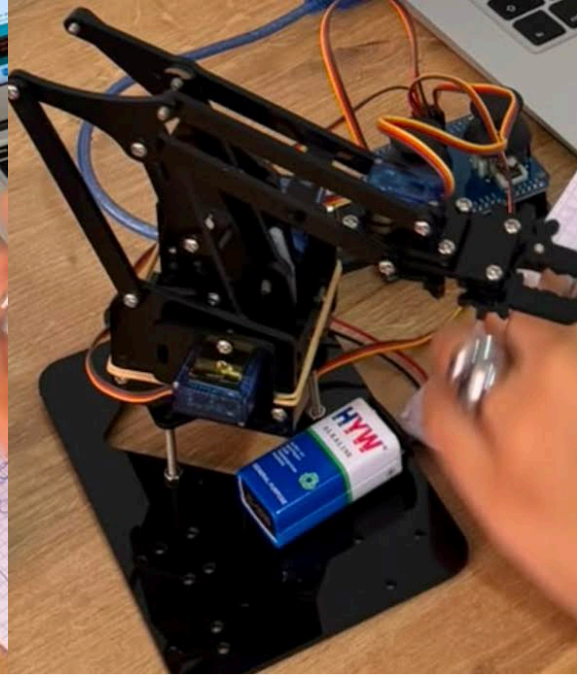
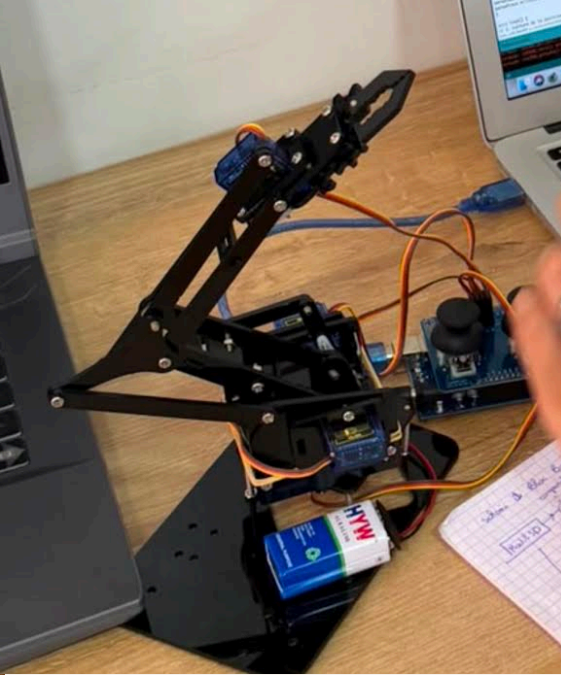
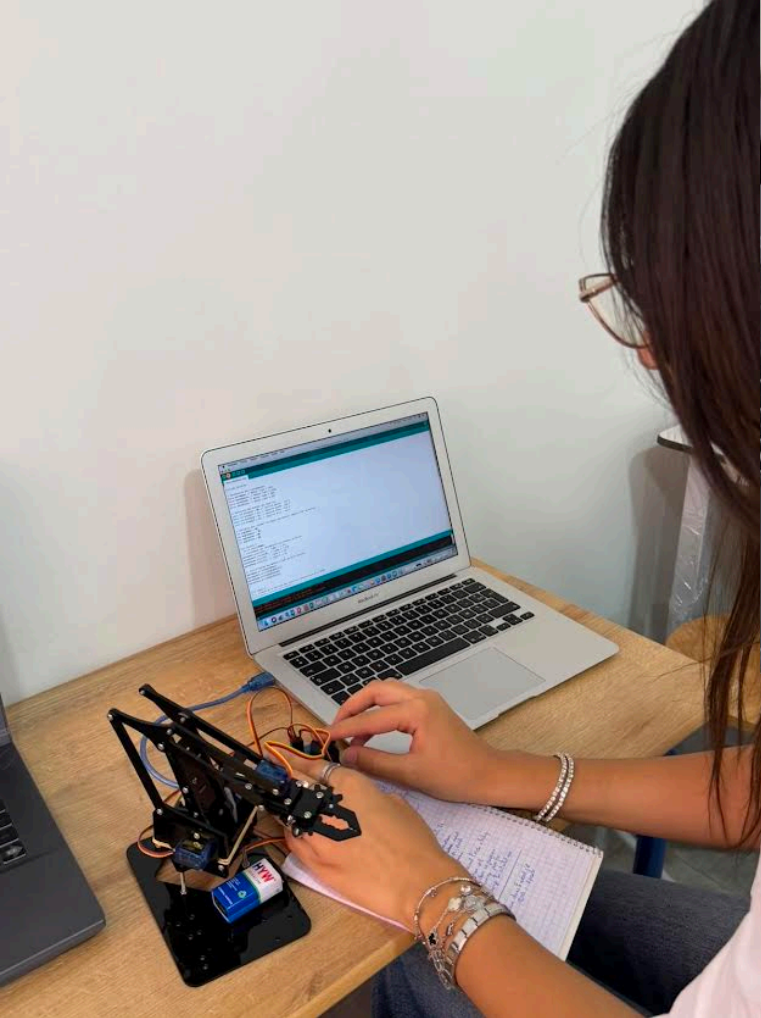


Etape 3: avancement de la construction



Etape 4: Dernière finition avant la programmation





ETAPE 5 : programmation de la carte arduino



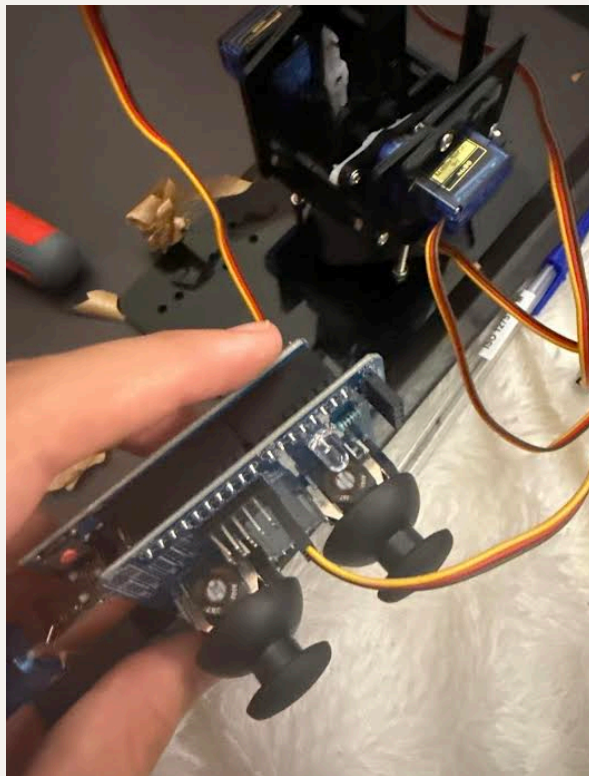
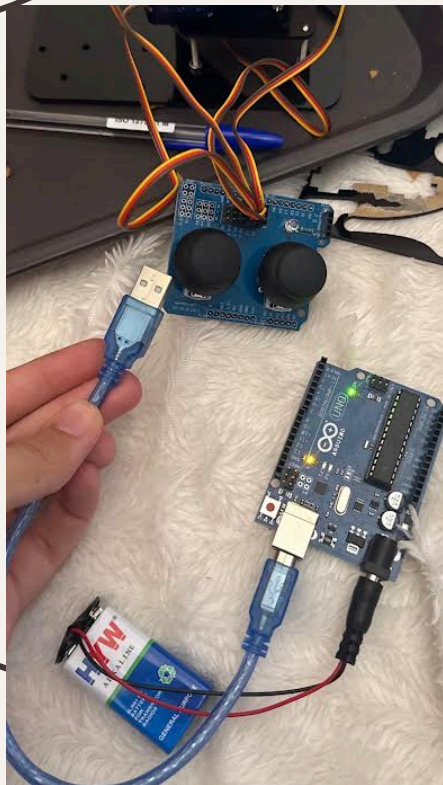
CONCLUSION



Figure 13

L'étude théorique et les validations expérimentales démontrent que le robot ROSA utilise ses asservissements en boucle fermée pour corriger en temps réels la position du bras , garantissant ainsi la précision micrométrique et la sécurité indispensables aux gestes chirurgicaux

MONTAGE ARDUINO



Commentaire :

Réalisation d'un prototype de commande pour valider l'algorithme de positionnement avant intégration finale

bras_robotique_rosa \$

```
#include <Servo.h>

// Déclaration des 4 servomoteurs
Servo servoBase; // Moteur ligne 1 (D11)
Servo servoEpaule; // Moteur ligne 2 (D10)
Servo servoCoude; // Moteur ligne 3 (D9)
Servo servoPince; // Moteur ligne 4 (D5)

// Définition des broches des Joysticks
const int pinJoy1X = A0; // Joystick gauche - Axe X
const int pinJoy1Y = A1; // Joystick gauche - Axe Y
const int pinJoy2X = A2; // Joystick droit - Axe X
const int pinJoy2Y = A3; // Joystick droit - Axe Y

// Variables pour stocker les angles des moteurs (départ à 90° au milieu)
int angleBase = 90;
int angleEpaule = 90;
int angleCoude = 90;
int anglePince = 90;

void setup() {
  Serial.begin(9600);
  // Attachement des servomoteurs aux broches du Shield
  servoBase.attach(11); // Ligne 1 -> D11
  servoEpaule.attach(10); // Ligne 2 -> D10
  servoCoude.attach(9); // Ligne 3 -> D9
  servoPince.attach(5); // Ligne 4 -> D5

  // Initialisation des moteurs à leur position centrale
  servoBase.write(angleBase);
  servoEpaule.write(angleEpaule);
  servoCoude.write(angleCoude);
  servoPince.write(anglePince);
}
```

```

void loop() {
// 1. Lecture de la position des joysticks (valeurs entre 0 et 1023)
int valJoy1X = analogRead(pinJoy1X);
int valJoy1Y = analogRead(pinJoy1Y);
int valJoy2X = analogRead(pinJoy2X);
int valJoy2Y = analogRead(pinJoy2Y);

// 2. Commande de la BASE (Joystick 1 - Axe X)
if (valJoy1X < 400) angleBase = constrain(angleBase + 1, 0, 180);
if (valJoy1X > 600) angleBase = constrain(angleBase - 1, 0, 180);

// 3. Commande de l'ÉPAULE (Joystick 1 - Axe Y)
if (valJoy1Y < 400) angleEpaule = constrain(angleEpaule - 1, 0, 180);
if (valJoy1Y > 600) angleEpaule = constrain(angleEpaule + 1, 0, 180);

// 4. Commande du COUDE (Joystick 2 - Axe Y)
if (valJoy2Y < 400) angleCoude = constrain(angleCoude - 1, 0, 180);
if (valJoy2Y > 600) angleCoude = constrain(angleCoude + 1, 0, 180);

// 5. Commande de la PINCE (Joystick 2 - Axe X)
if (valJoy2X < 400) anglePince = constrain(anglePince - 1, 10, 170);
if (valJoy2X > 600) anglePince = constrain(anglePince + 1, 10, 170);

// 6. Envoi des nouveaux angles aux moteurs
servoBase.write(angleBase);
servoEpaule.write(angleEpaule);
servoCoude.write(angleCoude);
servoPince.write(anglePince);

```

```

// Petite pause pour fluidifier le mouvement

```

```

delay(15);

```

```

// 7. Envoyer les angles au PC via USB

```

```

Serial.print(millis()); // Temps écoulé en ms ( Axe X de tes future

```

```

Serial.print(",");

```

```

Serial.print(angleBase);

```

```

Serial.print(",");

```

```

Serial.print(angleEpaule);

```

```

Serial.print(",");

```

```

Serial.print(angleCoude);

```

```

Serial.print(",");

```

```

Serial.println(anglePince);

```

```

}

```

```
import numpy as np
import matplotlib.pyplot as plt

# 1. PARAMÈTRES GLOBAUX
DT, T_TOTAL = 0.001, 5.0 # Boucle 1 kHz, durée 5s
N_STEPS = int(T_TOTAL / DT)
TIME = np.linspace(0, T_TOTAL, N_STEPS)
KP, KI, KD = 50.0, 5.0, 6.0
TAU, MASS = 0.015, 0.20
SENSOR_NOISE_STD = 0.015
PERT_AMP, PERT_FREQ = 0.08, 8.0
PRECISION_THRESHOLD = 0.5

# 2. TRAJECTOIRE CIBLE & DÉRIVÉE (FEEDFORWARD)
def reference(t):
    main = 10.0 / (1.0 + np.exp(-4.0 * (t - 1.5)))
    micro = 0.15 * np.sin(2 * np.pi * 0.4 * t) * (t > 2.5)
    return main + micro

def reference_dot(t):
    return 10.0 * np.exp(-4 * (t - 1.5)) / (1.0 + np.exp(-4 * (t - 1.5)))**2
```

```

# 3. BOUCLE DE SIMULATION PRINCIPALE (ROSA OPTIMAL)
pos, vel = 0.0, 0.0 # État initial du bras
integral, prev_error = 0.0, 0.0
filt_d, alpha = 0.0, 0.15 # Pour le PID
kf_x, kf_P, kf_Q, kf_R = 0.0, 1.0, 0.0005, SENSOR_NOISE_STD**2 # Pour Kalman

# Tableaux de stockage
arr_ref, arr_real, arr_meas, arr_err, arr_u, arr_pert = [np.zeros(N_STEPS) for _ in range(6)]

np.random.seed(42)
for i, t in enumerate(TIME):
    r = reference(t)
    r_dot = reference_dot(t)

    # Perturbation (Vibrations + pic à t=[3.0, 3.05])
    perturb = PERT_AMP * np.sin(2 * np.pi * PERT_FREQ * t)
    if 3.0 <= t <= 3.05:
        perturb += 0.30

    # Capteur + Filtre de Kalman
    z = pos + np.random.normal(0, SENSOR_NOISE_STD)
    kf_P_pred = kf_P + kf_Q
    K = kf_P_pred / (kf_P_pred + kf_R)
    kf_x = kf_x + K * (z - kf_x)
    kf_P = (1 - K) * kf_P_pred

    # Régulateur PID + Feedforward
    e = r - kf_x
    integral = np.clip(integral + e * DT, -8.0, 8.0)
    raw_d = (e - prev_error) / DT
    filt_d = alpha * raw_d + (1 - alpha) * filt_d
    prev_error = e

```

```

ff = MASS * r_dot * 2.0
u = np.clip(KP * e + KI * integral + KD * filt_d + ff, -100.0, 100.0)

# Modèle physique du bras (Intégration d'Euler)
acc = (u - (MASS / TAU) * vel + perturb) / MASS
vel = np.clip(vel + acc * DT, -2000, 2000)
pos = pos + vel * DT

# Sauvegarde des données
arr_ref[i], arr_real[i], arr_meas[i] = r, pos, kf_x
arr_err[i], arr_u[i], arr_pert[i] = r - pos, u, perturb

# 4. VISUALISATION (Similaire aux graphiques clés d'origine)
plt.figure(figsize=(12, 8))

# Graphique 1 : Trajectoire
plt.subplot(2, 2, 1)
plt.fill_between(TIME, arr_ref - PRECISION_THRESHOLD, arr_ref + PRECISION_THRESHOLD, color='g', alpha=0.1, label='Seuil')
plt.plot(TIME, arr_ref, 'y-', lw=2, label='Consigne IRM')
plt.plot(TIME, arr_real, 'g-', lw=1.5, label='Position Réelle')
plt.title('Trajectoire (Boucle fermée active)')
plt.grid(True, linestyle='--')
plt.legend()

# Graphique 2 : Erreur e(t)
plt.subplot(2, 2, 2)
plt.plot(TIME, arr_err, 'r-', label='Erreur e(t)')
plt.axhline(y=PRECISION_THRESHOLD, color='g', linestyle='--')
plt.axhline(y=-PRECISION_THRESHOLD, color='g', linestyle='--')
plt.title("Erreur de positionnement")
plt.grid(True, linestyle='--')
plt.legend()

```

```
# Graphique 3 : Commande u(t)
plt.subplot(2, 2, 3)
plt.plot(TIME, arr_u, 'b-', label='Commande u(t)')
plt.title('Signal de commande envoyé (Actionneur)')
plt.grid(True, linestyle='--')
plt.legend()

# Graphique 4 : Perturbations d(t)
plt.subplot(2, 2, 4)
plt.plot(TIME, arr_pert, 'm-', alpha=0.7, label='Tremor + Choc')
plt.title('Perturbations externes')
plt.grid(True, linestyle='--')
plt.legend()

plt.tight_layout()
plt.show()
```

Merci pour votre attention

